Лабораторная работа 3 PostgreSQL для сервиса

Части 1 и 2: разделы 1-5. Материал 05.

Студент: Есімханова Аружан
Группа: 25-ИСТК-2о
Дата: 22 октября 2026 года

Цель

Подключить DemoApi к настоящей PostgreSQL в контейнере, сохранить данные в именованном томе, проверить SQL-реализацию и соединить API с БД через пользовательскую сеть Docker.

Компонент	Фактическая конфигурация
База	PostgreSQL 18.6, postgres:18-alpine, Linux/amd64
Данные и инициализация	Именованный том + db/seed.sql, bind mount только для чтения
Путь данных PostgreSQL 18	/var/lib/postgresql/18/docker; том смонтирован в /var/lib/postgresql
Сервис Windows	localhost:8091 → PostgreSQL localhost:15432
Сервис в контейнере	localhost:8092 → API:8080; БД по DNS-имени контейнера:5432
Тесты	24 обязательных и 28 полных, PostgreSQL, отдельная база demo_tests

Результат

Технические проверки выполнены. Данные пережили пересоздание контейнера БД и перезапуск сервиса. Обе требуемые ошибки подключения воспроизведены. Включен настоящий скриншот Swagger. Для полного комплекта сдачи остаются скриншоты консоли Visual Studio и Test Explorer: запуск Windows проверен через dotnet, ручное F5 и интерфейс Test Explorer не подтверждены.

Версия лабораторной на GitHub

1. База без именованного тома

В отдельном учебном контейнере создана таблица notes и добавлена строка check. Контейнер удален и создан заново из того же образа с теми же параметрами пользователя и базы.

```
CREATE TABLE notes (text text); INSERT INTO notes VALUES ('check'); SELECT * FROM notes;
```

Фактический вывод первого контейнера:

```
CREATE TABLE
INSERT 0 1
check
```

Запрос во втором контейнере:

```
SELECT * FROM notes;
```

Фактический ответ:

```
ERROR:  relation "notes" does not exist
LINE 1: SELECT * FROM notes;
                      ^
```

Ответ на вопрос пункта 1: почему таблицы нет?

Новый контейнер не получил прежний каталог данных. Образ PostgreSQL объявляет VOLUME, поэтому даже без явного -v Docker создает анонимный том. У двух контейнеров разные анонимные тома; старый автоматически не подключается. Таблица находилась в прежней базе, а новая была инициализирована с нуля. Имена томов и разные ID контейнеров сохранены в доказательстве.

Важное различие

Само docker rm без -v не обязательно удаляет файлы анонимного тома: он может остаться без контейнера. Проблема эксперимента в отсутствии повторного подключения того же хранилища. После проверки были удалены только созданные для нее одноразовые контейнеры и их анонимные тома.

SQL, ответы и сведения о томах двух контейнеров

2. Именованный том и учебные данные

Для основной учебной БД создан том devops-labs-lab03-pgdata. Он подключен в /var/lib/postgresql; каталог проекта db подключен в /docker-entrypoint-initdb.d только для чтения. PostgreSQL опубликован исключительно на 127.0.0.1:15432.

Первый запуск: seed.sql создал 2 проекта и 4 задачи:

```
Курс DevOps|Установить Docker Desktop|done
Курс DevOps|Запустить БД в контейнере|in_progress
Курс DevOps|Написать Dockerfile|todo
Домашние дела|Купить продукты|todo
```

Затем через SQL добавлена «Задача из лабы 3». Контейнер БД удален и создан заново с тем же именованным томом. Фактический SELECT после пересоздания:

```
1|Установить Docker Desktop
2|Запустить БД в контейнере
3|Написать Dockerfile
4|Купить продукты
5|Задача из лабы 3
```

Журнал нового контейнера:

```
PostgreSQL Database directory appears to contain a database; Skipping initialization
```

Ответ на вопрос пункта 3: почему seed.sql не повторился?

Именованный том сохранил существующий кластер PostgreSQL, включая таблицы и добавленную задачу. Скрипты docker-entrypoint-initdb.d выполняются только при первоначальной инициализации пустого каталога данных. При наличии PGDATA образ пропускает инициализацию, поэтому 4 исходные строки не дублируются, а пятая сохраняется.

Пароль и путь данных

Пароль сгенерирован локально и не публикуется. Изменение переменной POSTGRES_PASSWORD на непустом томе не меняет пароль уже созданного пользователя. Для PostgreSQL 18 используется новый путь тома /var/lib/postgresql; это отличается от старых примеров PostgreSQL 17.

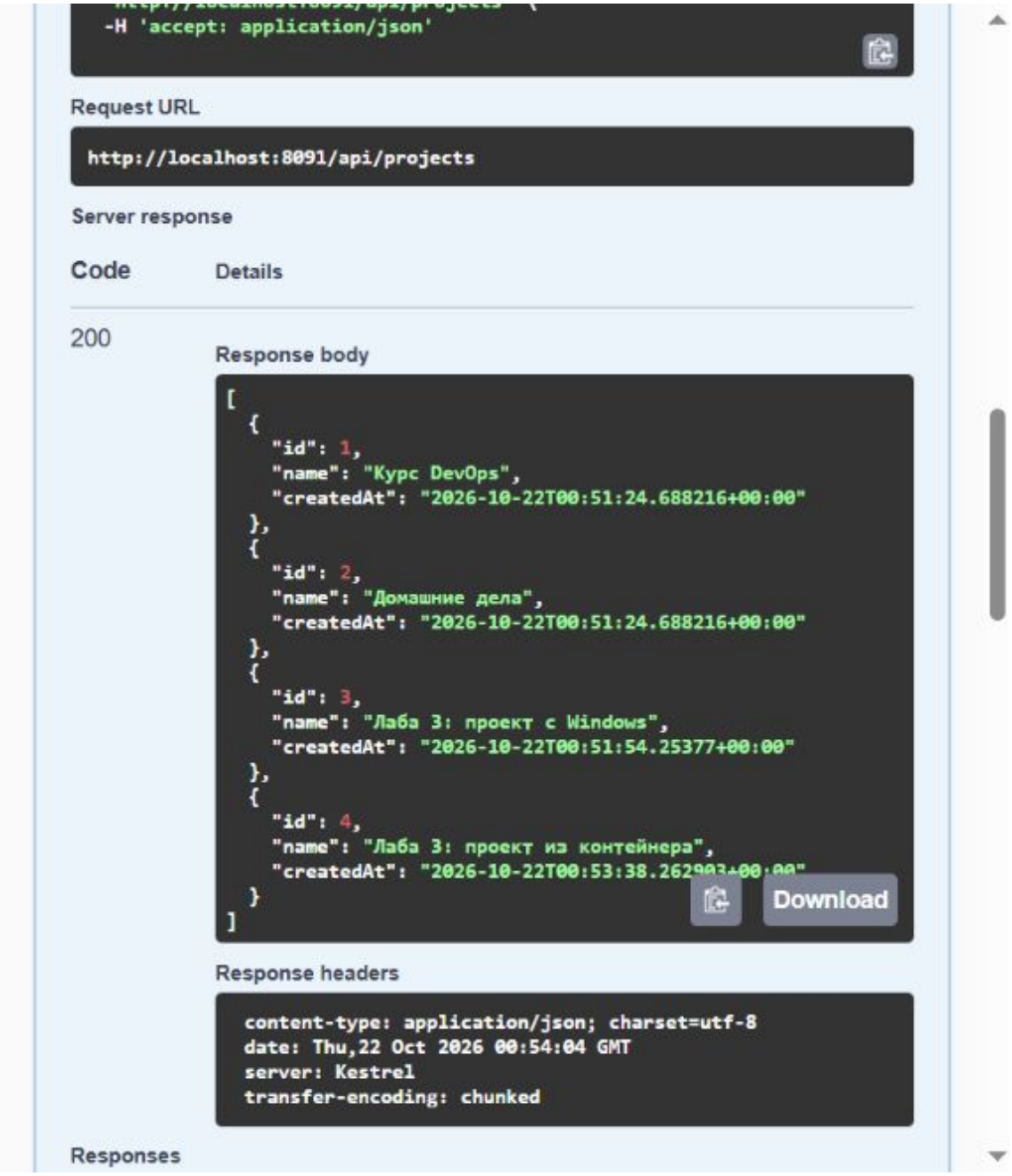
Доказательство пересоздания и сохранения задачи · Официальный образ PostgreSQL

3. Сервис Windows и сохранение данных

Сервис Windows запущен с Storage=Postgres и строкой подключения через переменные окружения. Через HTTP API создан проект; после завершения процесса и повторного запуска его ID и имя сохранились. Swagger и сервис-контейнер показывают тот же проект.

```
Хранилище: PostgreSQL
Now listening on: http://127.0.0.1:8091
Hosting environment: Development
```

Рисунок 1. Настоящий Swagger UI после перезапуска: HTTP 200, учебные проекты, проект Windows (id=3) и проект контейнера (id=4).



Снимок подтверждает ответ API. Скриншот консоли Visual Studio и запуск через F5 этим снимком не подменяются; текст настоящего журнала процесса приведен выше.

4. Тесты и SQL-статистика

Существующие тесты выполнены с Storage=Postgres и ConnectionStrings__Db, указывающей на отдельную базу demo_tests в этом же PostgreSQL. Код контроллеров, сервисов и SQL-репозиториев не менялся.

Запуск	Фактический результат
Обязательный набор	24/24: 7 юнит и 17 интеграционных; 0 failed, 0 skipped
Текущая версия полностью	28/28: 7 юнит и 21 интеграционный; 0 failed, 0 skipped
Сохраненные HTTP-сценарии	26 успешных запросов: 23 сценария и 3 удаления тестовых проектов
Postman CLI	24 запроса, 73 проверки, 0 ошибок; код завершения 0

Как получены именно 24 случая

В текущей версии есть четыре дополнительных случая проверки срока задачи. Первый запуск исключает их фильтром; второй выполняет все 28. Это обязательная выборка текущего кода, а не подмена итогового числа тестов.

```
FullyQualifiedName!~Create_PastDueDate&FullyQualifiedName!~Create_ValidDueDate
```

SQL-версия статистики

Два интеграционных теста статистики прошли на PostgreSQL: подсчет задач и просрочек, пустой проект, срок сегодня/null, изоляция проектов, завершенные задачи и неизвестный проект. Исправление ProjectRepository.GetStatsAsync не потребовалось.

Дополнительно: GET /api/projects/1/stats на учебных данных с добавленной пятой задачей:

```
{
  "projectId": 1,
  "total": 4,
  "todo": 2,
  "inProgress": 1,
  "done": 1,
  "overdue": 0
}
```

Сводки извлечены из настоящих TRX; локальные имена машины и пользователя исключены. Скриншот 24 зеленых случаев в Test Explorer пока отсутствует. Postman проверял реальный контейнер API; результаты в облако не публиковались.

5. Три попытки подключения контейнера

Попытка 1: задано только Storage=Postgres

```
Unhandled exception. System.InvalidOperationException: Не задана строка подключения
ConnectionStrings:Db
```

Причина: по умолчанию контейнер запускается в Production. Строка в appsettings.Development.json в этом окружении не загружается. Без ConnectionStrings__Db приложение не может создать источник данных и завершается.

Попытка 2: строка с Host=localhost

```
Unhandled exception. Npgsql.NpgsqlException (0x80004005): Failed to connect to 127.0.0.1:5432
 ---> System.Net.Sockets.SocketException (111): Connection refused
```

Причина: localhost внутри API-контейнера указывает на сам API-контейнер. PostgreSQL работает в другом контейнере, поэтому на 127.0.0.1:5432 нет слушателя. Обе попытки действительно завершились с ненулевым кодом; журналы сохранены.

Попытка 3: общая сеть и DNS-имя БД

```
Storage=Postgres
ConnectionStrings__Db=Host=devops-labs-lab03-db;Database=demo;
    Username=demo;Password=<локальный пароль>
```

Проверка	Результат
Сеть	devops-labs-lab03-net, 10.254.53.0/24
Разрешение DNS	Имя БД разрешено внутри API-контейнера
Запуск и готовность	Контейнер healthy, /health/ready → 200
Общие данные	Виден проект Windows id=3
docker stop/start API	Проект контейнера id=4 сохранился

Отличие рабочей среды

Стандартные пулы подсетей Docker на этой машине исчерпаны. Для отдельной учебной сети выбрана явная подсеть после проверки пересечений с сетями Docker и маршрутами Windows. Другие проекты не изменялись. Порт Windows 15432 используется только для Windows-клиентов; контейнеры общаются напрямую на 5432.

6. Ответы и итог сдачи

Почему том сохраняет данные?

Том существует отдельно от контейнера. Новый контейнер с тем же томом получает тот же кластер PostgreSQL. Это проверено пересозданием контейнера и сохранением пятой задачи.

Чем том отличается от bind mount?

Именованным томом управляет Docker; здесь в нем данные БД. Bind mount подключает конкретный каталог Windows; здесь это db со скриптом seed.sql, подключенный только для чтения.

Почему seed.sql не запускается повторно?

Entrypoint официального образа выполняет init-скрипты только при пустом PGDATA. Именованный том уже содержит базу, поэтому журнал сообщает Skipping initialization.

Почему адрес БД зависит от места запуска API?

Windows использует опубликованный localhost:15432. API-контейнер использует DNS-имя БД и внутренний порт 5432 общей сети. Его localhost не является адресом Windows или соседнего контейнера.

Что переживает Compose down и down -v?

По назначению Compose down удаляет контейнеры и сеть, сохраняя именованные тома; down -v удаляет и управляемые тома с данными. Команды Compose в материале 05 не выполнялись: это следующее задание 06.

Итог

Выполнены части 1 и 2 лабораторной 3: PostgreSQL, учебные данные, том, сервис Windows, тесты, две ошибки подключения и успешный запуск контейнера в общей сети. Код приложения сохранил прежнее поведение; SQL-реализация впервые подтверждена реальными запусками.

Для полного комплекта нужны скриншоты консоли Visual Studio со строкой «Хранилище: PostgreSQL» и Test Explorer с 24 зелеными случаями. Фактические консольные журналы и результаты есть, но прохождение этих UI-пунктов не объявляется подтвержденным.

После проверок API и БД остановлены. Именованный том с данными сохранен для следующей работы; локальный .env исключен из Git. Действия в СДО остаются за студентом.

Источники и материалы

Материал 05: «Неделя 5 - Контрольная точка 3», лабораторная 3, части 1-2 и лекция 3 из загруженного архива Devops (1).zip; kit.zip распакован локально.

Инструкция и ответы · Журналы и доказательства · Параметры Npgsql